

Keywords:

- AVX: floating point only
- AVX2: floating + integer
- AVX-512: adds per-lane masking, scatter/gather, and more registers (zmm0–zmm31)
- <https://arxiv.org/pdf/2406.04693v1>

Code Vectorization:

Vectorization is the process of transforming scalar operations (one element at a time) into vector operations (many elements per instruction) using SIMD hardware like SSE, AVX, or your own vector core.

1. No data dependencies between iterations (e.g. no $a[i] = a[i-1] + b[i]$).
2. Memory alignment or safe unaligned access.
3. Predictable loop bounds and no hidden side effects.

Register Files (Architectural View):

- SSE: 16×128 -bit registers (xmm0–xmm15)
- AVX/AVX2: 16×256 -bit registers (ymm0–ymm15)
- AVX-512: 32×512 -bit registers (zmm0–zmm31) + mask registers (k0–k7)

Each register can hold multiple scalar values:

- ymm (256 bits) can hold 8×32 -bit floats → 8-way parallelism
- zmm (512 bits) can hold 16×32 -bit floats → 16-way parallelism

Intrinsics: The Programmer's Access to SIMD

```
#include <immintrin.h>

void add_arrays(float* a, float* b, float* c) {
    __m256 va = _mm256_loadu_ps(a); // load 8 floats
    __m256 vb = _mm256_loadu_ps(b);
    __m256 vc = _mm256_add_ps(va, vb); // vector add
    _mm256_storeu_ps(c, vc);           // store result
}
```

Compiler Auto-Vectorization

Compilers (like GCC, Clang, ICC) can automatically generate SIMD instructions if:

- The loop has no data dependencies between iterations.
- Memory is aligned or the compiler can prove no aliasing.

Example: Loop Vectorization with AVX2

```
for (int i = 0; i < 8; i++)  
    c[i] = a[i] * b[i] + d[i];
```

```
vmovups ymm0, [a]      ; load 8 floats  
vmovups ymm1, [b]  
vmovups ymm2, [d]  
vmulps ymm3, ymm0, ymm1 ; multiply  
vaddps ymm4, ymm3, ymm2 ; add  
vmovups [c], ymm4      ; store
```

Suggestions From Professor Wang:

1. Implement packetization in source code (C/C++ stage)
2. Slightly harder to implement at IR stage but doable (Personally recommended)
3. DO NOT implement at assembly stage (very difficult in tracking jumps and loops)

Instruction Dependencies:

Data Dependencies

- **True Dependency** - RAW (Read After Write)
 1. For each instruction, record the register it reads from and writes to
 - DEF set: registers or memory locations it writes

- USE set: registers or memory locations it reads
- Ex.

- I1: $r1 = r2 + r3$
 - $DEF(I1) = \{r1\}$
 - $USE(I1) = \{r2, r3\}$
- I2: $r4 = r1 + r5$
 - $DEF(I2) = \{r4\}$
 - $USE(I2) = \{r1, r5\}$

2. For each pair (I1, I2) where I1 precedes I2:

- $RAW \rightarrow DEF(I1) \cap USE(I2) \neq \emptyset$
- $WAR \rightarrow USE(I1) \cap DEF(I2) \neq \emptyset$
- $WAW \rightarrow DEF(I1) \cap DEF(I2) \neq \emptyset$
- If any intersection is nonempty, there's a dependency edge $I1 \rightarrow I2$.

3. Each instruction becomes a node; each dependency becomes a directed edge.

- Ex.

- I1: $r1 = r2 + r3$

I2: $r4 = r1 + r5$

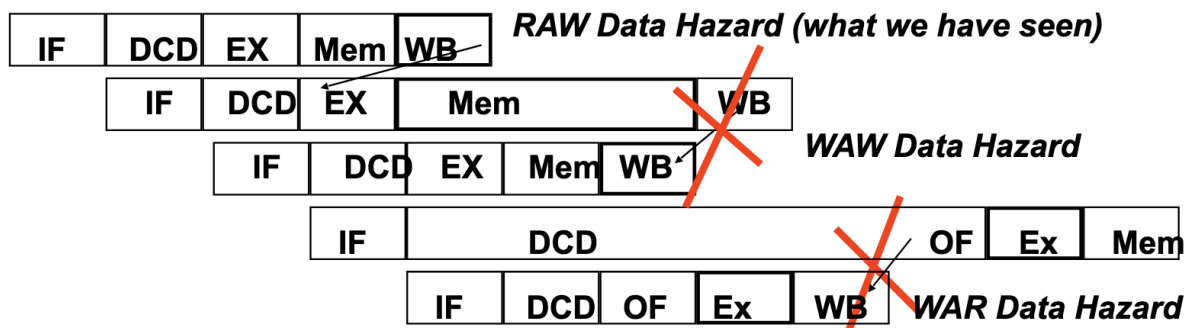
I3: $r6 = r7 + r8$

- Edges: $I1 \rightarrow I2$ (RAW on r1).
- I3 is independent.
- So I1 and I3 can go in the same packet; I2 must wait.

4. Once you have the dependency graph:

- Perform a topological sort of the graph to find valid execution order.
- For each instruction, collect all ready nodes (no unsatisfied predecessors).
- Fill a packet with as many ready instructions as the hardware allows (e.g., 4 ops per cycle).
- Remove them from the graph, repeat.

- **Might Confuse Compiler** - WAR (Write After Read) & WAW (Write After Write)



Memory Dependencies

Type	Meaning	Example	Effect
RAW (Read After Write)	Load depends on an earlier store	STORE [A], X → LOAD [A]	Must serialize
WAR (Write After Read)	Store after load, same address	LOAD [A] → STORE [A]	Serialize to preserve order
WAW (Write After Write)	Two stores to same address	STORE [A], X → STORE [A], Y	Last store must appear last

- Unlike registers, addresses can come from computed pointers or base + offset expressions → Need **Aliasing**
- 1. Represent each memory access
 - For every instruction, compute:
 - Base address register (if any)
 - Constant offset
 - Access size (e.g. 4 bytes)
 - Whether it's a load or store
- 2. Compare pairs of accesses
 - Check if two memory references may overlap
 - Overlap = (base1 == base2) and ranges_intersect(offset1, size1, offset2, size2)
 - If yes:
 - If one is a write → they are dependent.
 - Otherwise, they're independent and can execute in parallel.
 - No overlap → safe to parallelize.
- Unknown / pointer-based addresses
 - When addresses are computed indirectly (e.g., from variables or function parameters), the compiler uses alias analysis.

Alias type	Meaning	Safe to reorder?
NoAlias	Proven disjoint	✓
MayAlias	Possibly overlap	✗ conservatively serialize
MustAlias	Definitely same	✗ must serialize

1. If two addresses have different base registers → assume NoAlias (safe).
2. If they share base but different constant offsets with no overlap → NoAlias.

3. If base or offset unknown → MayAlias → treat as dependent.

Control Dependencies

- A control dependency arises when an instruction's execution depends on the outcome of a branch (or any control-flow decision).
- Compilers use the Control Flow Graph (CFG), where:
 - Each node = basic block (straight-line code with no internal jumps)
 - Each edge = possible branch between blocks
- Then they compute a Post-Dominator Tree (PDT).
 - A node X dominates Y if every path from the entry to Y passes through X.
 - A node X post-dominates Y if every path from Y to the exit passes through X.
- An instruction (or block) B is control-dependent on a branch in block A if:
 - B post-dominates some but not all successors of A, and
 - B does not post-dominate A.
- Implementation
 1. Build a Control Flow Graph (CFG) for the function.
 2. Compute the Post-Dominator Tree (you can adapt a standard algorithm).
 3. For each branch node A:
 - For each successor S of A:
 - If S does not post-dominate A:
 - All blocks in the path from S to A's post-dominator are control-dependent on A.